



Bootloader optimizations





Generic bootloader optimizations



Bootloader

- ▶ Remove unnecessary functionality. Usually, bootloaders include many features needed only for development. Compile your bootloader with fewer features.
- ▶ Optimize required functionality. Tune your bootloader for fastest performance. Skip the bootloader and load the kernel right away.



U-Boot - Remove unnecessary functionality

Recompile U-Boot to remove features not needed in production

- ▶ Disable as many features as possible through the `menuconfig` interface and through `include/configs/<soc>-<board>.h`
- ▶ Examples: MMC, USB, Ethernet, dhcp, ping, command line edition, command completion...
- ▶ A smaller and simpler U-Boot is faster to load and faster to initialize.

However, in this presentation, we will give the easiest optimizations in U-Boot, but won't be exhaustive, because the best way to save time is to skip U-Boot, using its *Falcon Mode* (covered in the next section).



U-Boot - Remove the boot delay

- ▶ Remove the boot delay: `setenv bootdelay 0`
- ▶ This usually saves several seconds!



U-Boot - Simplify scripts

Some boards have over-complicated scripts:

```
bootcmd=run bootf0  
bootf0=run ${args0}; setenv bootargs ${bootargs} \  
maximasp.kernel=maximasp_nand.0:kernel0; nboot 0x70007fc0 kernel0
```

Running nested scripts

Let's replace this by:

```
setenv bootargs 'mem=128M console=tty0 consoleblank=0  
console=ttyS0,57600  
mtdparts=maximasp_nand.0:2M(u-boot)ro,512k(env0)ro,512k(env1)ro,  
4M(kernel0),4M(kernel1),5M(kernel2),100M(root0),100M(root1),-(other)  
rw ubi.mtd=root0 root=ubi0:rootfs rootfstype=ubifs earlyprintk debug  
user_debug=28 maximasp.board=EEKv1.3.x  
maximasp.kernel=maximasp_nand.0:kernel0'  
setenv bootcmd 'nboot 0x70007fc0 kernel0'
```

This saved 56 ms on this ARM9 system (400 MHz)!



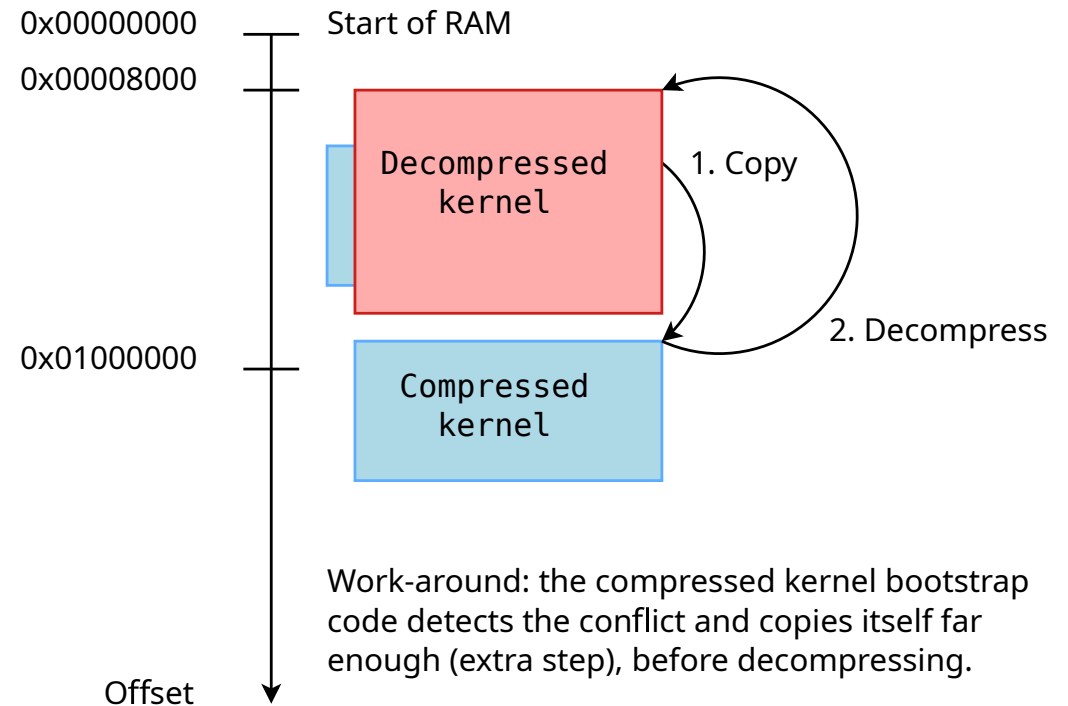
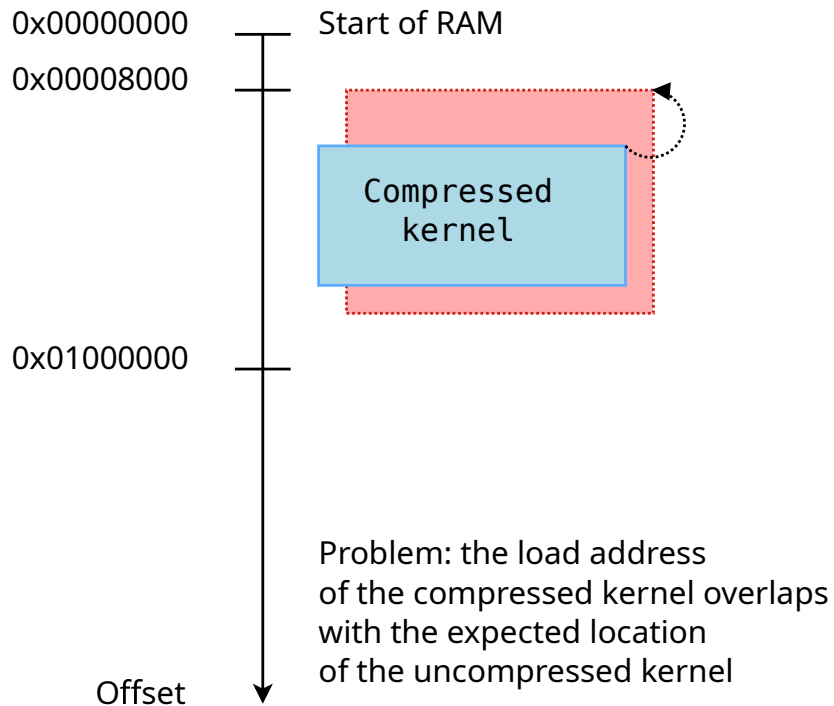
Bootloader: copy the exact kernel size

- ▶ When copying the kernel from **raw** flash or MMC to RAM, we still see many systems that copy too many bytes, not taking the exact kernel size into account.
- ▶ A solution is to store the exact size of the kernel in an environment variable, and use it a kernel loading time.
- ▶ Of course, that's not needed when the kernel is loaded from a filesystem, which knows how big the file is.



Bootloader: watch the compressed kernel load address

On ARM32, the uncompressed kernel is usually started at offset 0x8000 from the start of RAM. Load the compressed kernel at a far enough address!

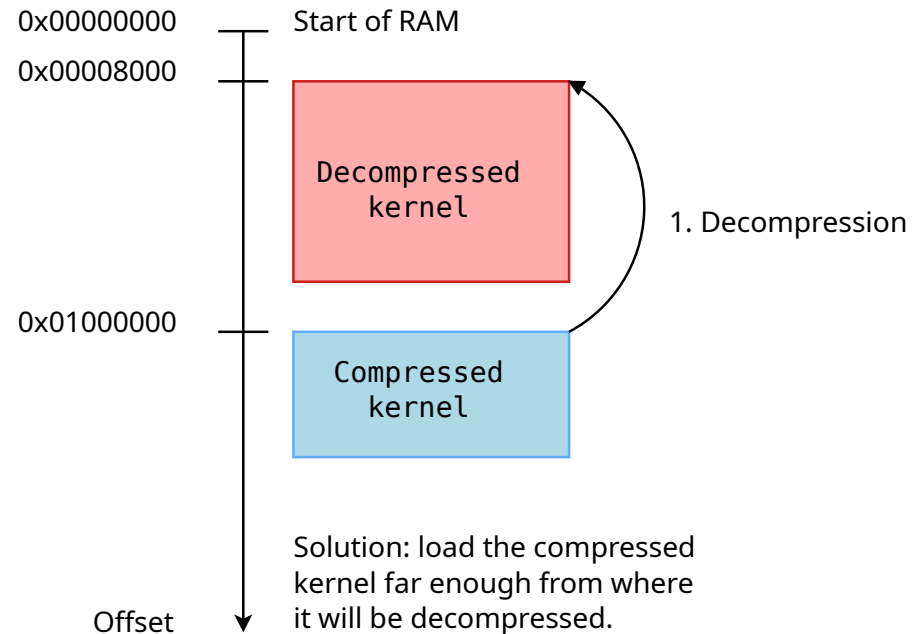


Source: <https://people.kernel.org/linusw/how-the-arm32-linux-kernel-decompresses>



Bootloader: load the compressed kernel far enough

On ARM32, a usual kernel load address is at offset 0x01000000 (16 MB)



Tests on STM32MP157A (650 MHz): an overlap increases boot time by 107 ms.